

Unsupervised Phenotype Discovery from Electronic Health Records

Computational Medicine · Assignment handout · Tensor factorization of clinical records

Deadline	11 Oct 2026 11:59pm
Weight	6% of the final grade
Submission	A single notebook, executed in full, with all output visible
Data	ehr_records.csv and patient_groups.csv, distributed with this handout
Tasks	Completion of 25 blanks across five functions; seven concept questions; six written questions
Verification	Automated. check_all() must report 5 of 5 finished in the submitted notebook.
Reference	Wang Y, Chen R, Ghosh J, Denny JC, Kho A, Chen Y, Malin BA, Sun J. Rubik: Knowledge Guided Tensor Factorization and Completion for Health Data Analytics. <i>KDD</i> 2015, 1265–1274.

1. Background

Electronic health record systems contain very large numbers of entries of the following form:

patient 41 · type 2 diabetes · metformin

An individual entry conveys little information. In aggregate, however, such records exhibit structure: particular diagnoses and medications co-occur systematically within subsets of patients. Clinically coherent subgroups of this kind are referred to as **phenotypes**.

The conventional approach to phenotyping relies on rules specified by clinicians, which require substantial expert effort for each condition and do not scale to an entire clinical database. This assignment examines whether phenotypes can instead be recovered **without supervision**, that is, without labelled examples and without prior specification of the groups to be found.

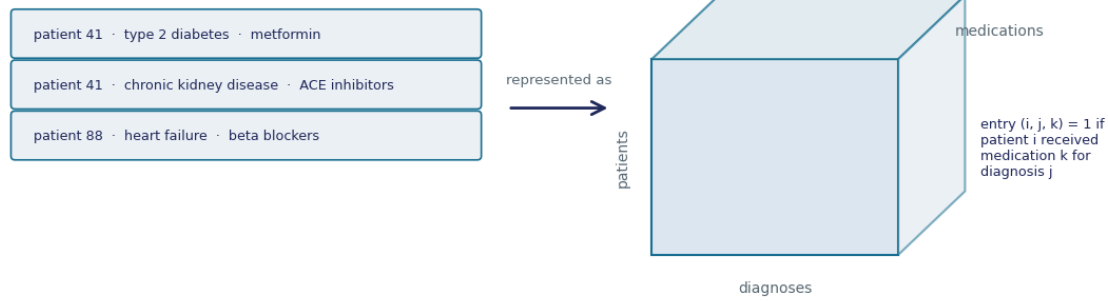
2. Method

2.1 Tensor representation

A tensor is a multidimensional array; the number of indices required to identify a single entry is its **order**. Each record specifies a patient, a diagnosis and a medication, and the records are therefore represented as a third-order tensor. Each dimension is termed a **mode**.

Entry (i, j, k) equals 1 if patient i received medication k for diagnosis j , and 0 otherwise. Because no patient presents with every condition, the tensor is highly sparse.

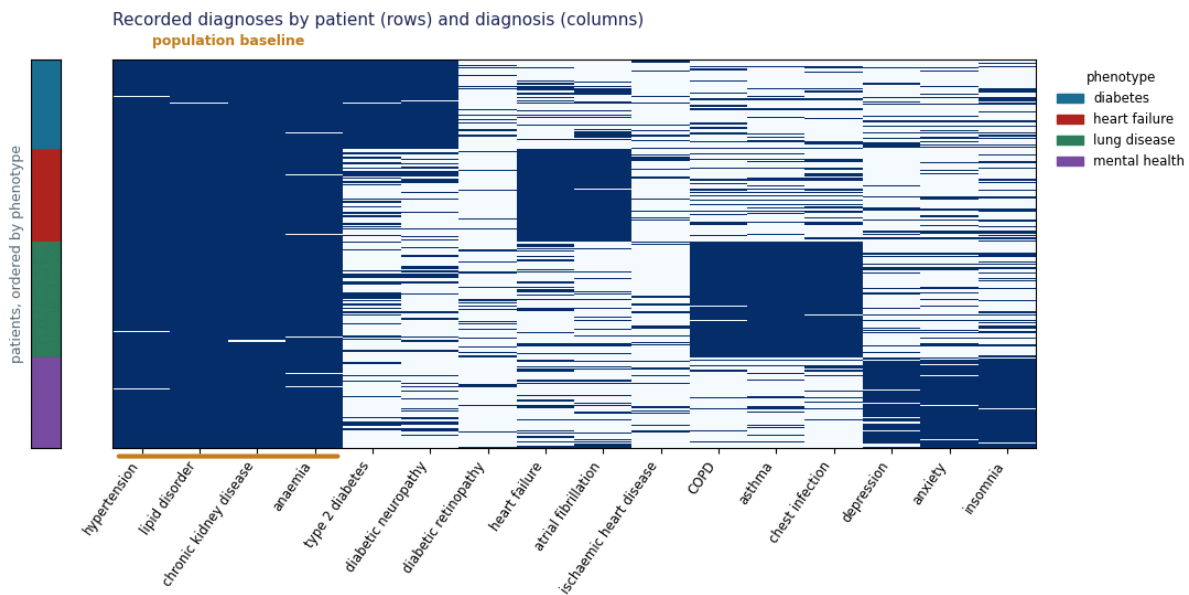
Individual records



A third-order tensor: a three-dimensional array indexed by patient, diagnosis and medication.

2.2 The dataset

The figure below presents the data used in this assignment as a matrix of patients (rows) against diagnoses (columns), in which an entry is shaded where the diagnosis is recorded. Patients are ordered by phenotype membership.

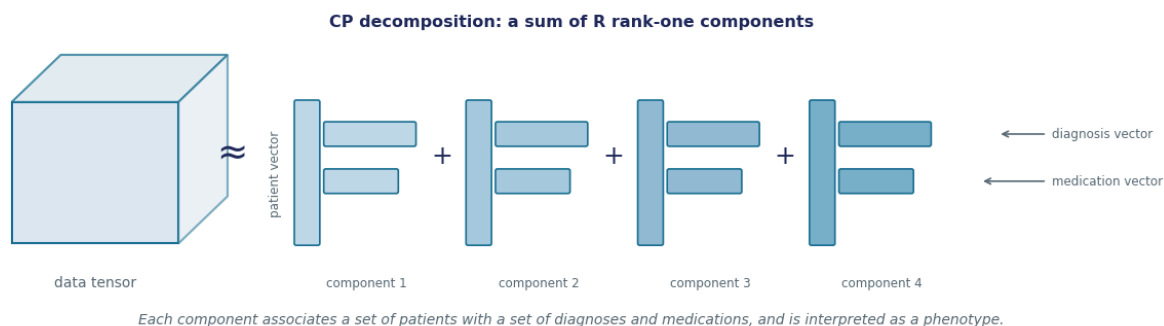


The four leftmost columns are populated for almost every patient. These columns constitute the **population baseline**, which is therefore apparent before any analysis is performed. Its significance is discussed in Section 2.4.

2.3 CP decomposition

A **rank-one tensor** is the outer product of one vector per mode, $a \otimes b \otimes c$, with entries $T(i,j,k) = a_i b_j c_k$. The **CP decomposition** approximates the data tensor as a sum of R such components.

Each component comprises a patient vector, a diagnosis vector and a medication vector, and is interpreted as one phenotype. A component assigning high weight to type 2 diabetes and diabetic neuropathy, and to insulin and metformin, may be interpreted as a diabetes phenotype. This association is not specified in advance; it emerges from the co-occurrence of these features in the records. The number of components R is specified by the analyst, analogously to the number of clusters in k-means.



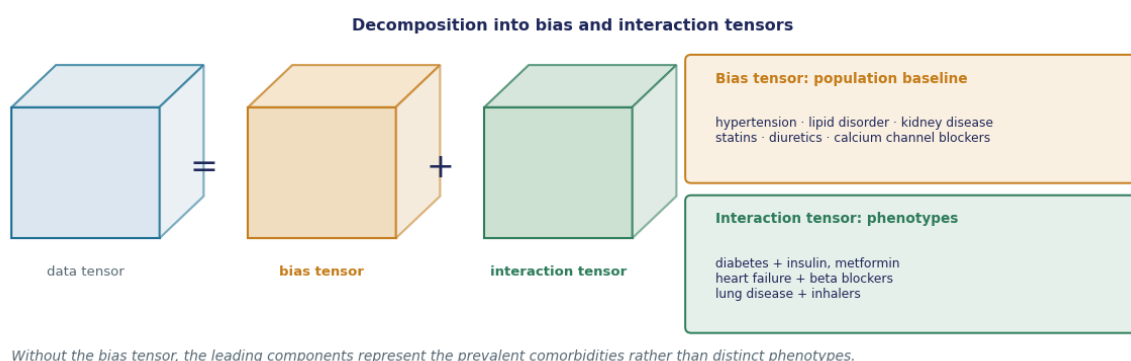
2.4 The bias tensor

A standard decomposition of clinical records exhibits a characteristic failure. Highly prevalent comorbidities, such as hypertension and lipid disorders together with their treatments, are present in almost every patient and therefore constitute the dominant pattern in the data. The decomposition consequently allocates a component to them. Such a component does not represent a phenotype, and the number of components remaining is insufficient to recover every genuine phenotype.

Rubik resolves this difficulty by modelling the data as the sum of two terms:

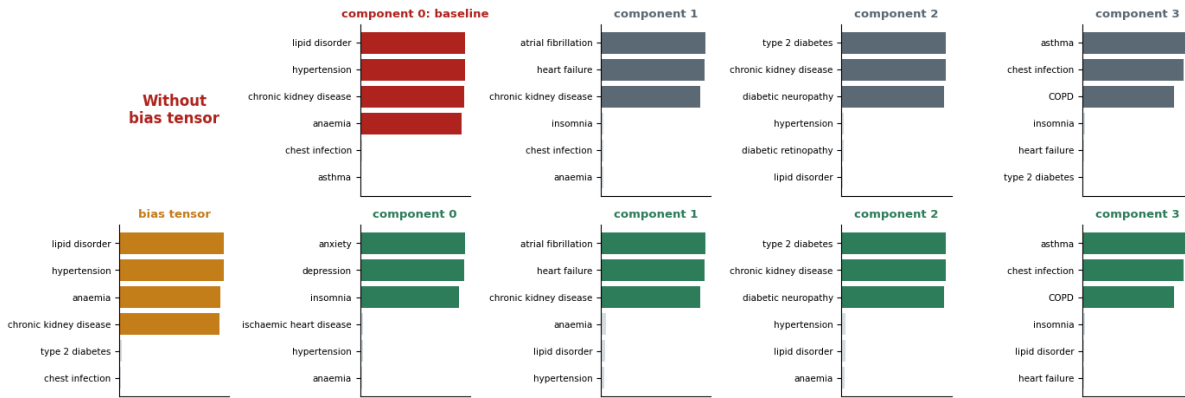
$$\mathcal{X} \approx \mathcal{C} + \mathcal{I}$$

The **bias tensor** $\mathcal{C}$ is a single rank-one component representing the population baseline. The **interaction tensor** $\mathcal{I}$ is a sum of R rank-one components representing the phenotypes, that is, the patterns that distinguish subgroups of patients from the baseline.



The effect on the present data is shown below. Without the bias tensor (upper row), one component absorbs the baseline and one phenotype is not recovered. With the bias tensor (lower row), every component of the interaction tensor corresponds to a planted phenotype.

Upper row: standard factorization, one component absorbs the baseline. Lower row: with the bias tensor, every component is a phenotype.



2.5 Rationale

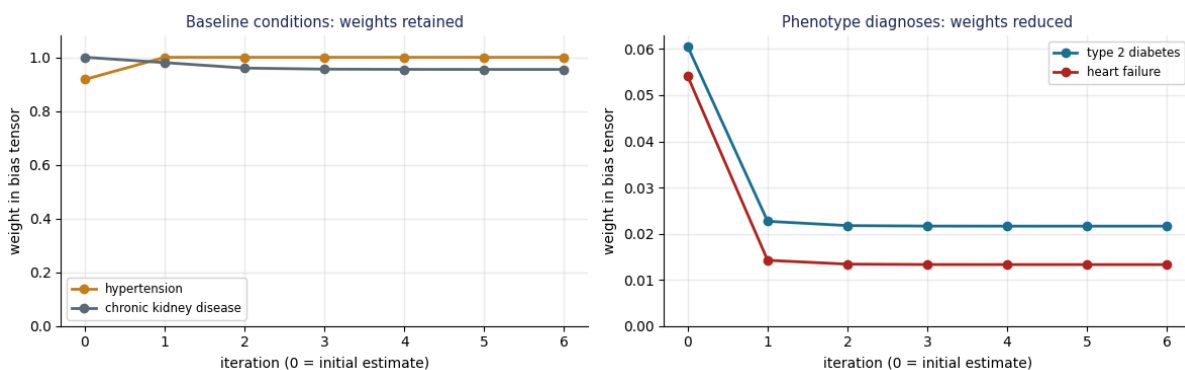
An analogy may clarify the purpose of the decomposition. Consider the task of describing a class photograph in which every pupil wears the same uniform. A description of the photograph as a whole would be dominated by the uniform, as it is the most prevalent feature, and the features that distinguish individual pupils would receive little attention. A more informative description first states that all pupils wear the uniform, and then describes only the features that differ. The uniform corresponds to the bias tensor; the distinguishing features correspond to the interaction tensor.

2.6 Estimation by alternating optimisation

The two terms are mutually dependent. An accurate estimate of the baseline requires knowledge of which features belong to the phenotypes, so that they can be excluded; conversely, the phenotypes can be identified only once the baseline is known. The terms are therefore estimated alternately:

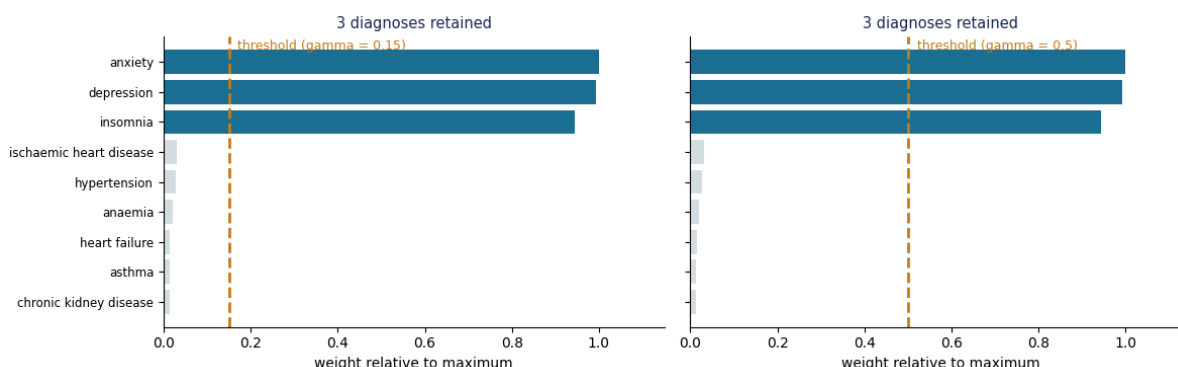
1. Estimate the bias tensor from the complete data. This initial estimate partially absorbs signal belonging to the phenotypes.
2. Subtract the bias tensor, and estimate the interaction tensor from the residual.
3. Subtract the interaction tensor, and re-estimate the bias tensor from the residual, which is now largely free of phenotype signal.
4. Repeat steps 2 and 3 for a fixed number of iterations.

The figure below traces the weights of selected diagnoses in the bias tensor across iterations. The weights of baseline conditions (left) are retained, whereas those of phenotype diagnoses (right) decrease by more than half after a single iteration, indicating the transfer of phenotype signal from the bias tensor to the interaction tensor.



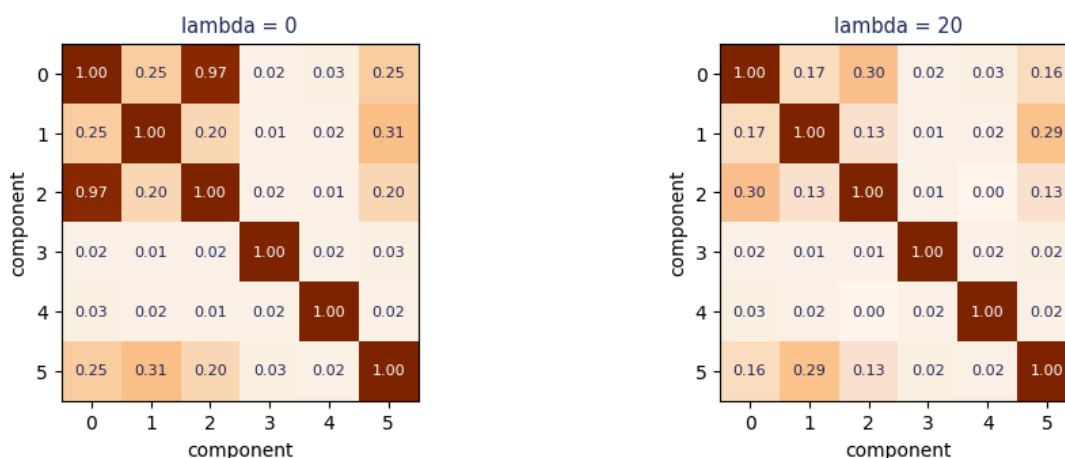
2.7 Sparsity and diversity

Sparsity. The factor vectors are dense: every diagnosis receives a non-zero weight in every component. Entries below a threshold are therefore set to zero, so that each phenotype is summarised by a short list. In this assignment the threshold is relative: an entry is retained only if it is at least a fraction γ of the largest entry in its vector.



Diversity. When R exceeds the number of underlying phenotypes, some components become near-duplicates. Rubik adds to the objective a penalty favouring mutually dissimilar components, weighted by λ . Redundancy is quantified by **AvgOverlap**, the mean cosine similarity over all pairs of components. The figure below shows the pairwise similarities without and with the penalty; the informative values lie off the diagonal.

Pairwise cosine similarity between components. Diagonal entries equal 1 by definition.



A lower overlap is not equivalent to a more accurate result. Genuine phenotypes may share features; in these data, the diabetes and heart failure phenotypes both include chronic kidney disease. A sufficiently large penalty will separate such components regardless, and may thereby conceal true structure. The penalty improves interpretability; it does not, in itself, improve validity.

3. Data

The records used in this assignment are synthetic by design. In real clinical data the true phenotypic structure is unknown, and the quality of a recovered phenotype can be assessed only by expert judgement; the original study required blinded review by three physicians for this reason. Planting known phenotypes in synthetic data permits exact, quantitative evaluation of recovery.

File	Contents
ehr_records.csv	One row per (patient, diagnosis, medication) triple; 5,817 rows
patient_groups.csv	The phenotype membership of each patient; 300 rows

In Colab, both files should be uploaded through the file panel; the notebook prompts for any file that is absent.

The data comprise 300 synthetic patients, 16 diagnoses and 14 medications, and contain:

- a **population baseline** exhibited by most patients: hypertension, lipid disorder, chronic kidney disease and anaemia, treated with statins, loop diuretics and calcium channel blockers; and
- **four planted phenotypes**, namely diabetes, heart failure, lung disease and mental health, with each patient belonging to one or two of them.

4. Tasks

The code for each function is provided in full, with the exception of a small number of blanks marked **FILL_IN**. Each blank must be replaced with an appropriate value: either a numerical constant, such as 0 or 1, or the name of an existing variable, such as `gamma` or `bias`. No additional lines of code are required. Each blank corresponds to a specific methodological decision. The code of each function is reproduced below, together with the questions that determine its blanks and a summary of the properties verified by its tests.

TODO 1 — reconstruct

Reconstruction of the tensor from its three factor matrices.

```
A, B, C = factors
return np.einsum("ir,jr,kr->ijk", FILL_IN, FILL_IN, FILL_IN)
```

Questions. The factor matrices A, B and C correspond to the patient, diagnosis and medication modes. In which order must they be supplied to match the subscripts "ir,jr,kr"?

Verified properties. The reconstructed tensor has the correct shape; each entry equals the product of the corresponding factor entries; and multiple components are summed independently.

TODO 2 — sparsify and top_items

Application of the sparsity threshold, and conversion of a factor vector to a list of labels.

```
v = vec.copy()
cut = FILL_IN * FILL_IN
v[v < cut] = FILL_IN
return v
```

```
v = sparsify(FILL_IN, FILL_IN)
order = np.argsort(FILL_IN)
return [labels[i] for i in order if v[i] > FILL_IN]
```

Questions. By which quantity is `gamma` multiplied to obtain the threshold? What value is assigned to entries below it? Since `argsort` orders values in ascending order, how is descending order obtained? Which labels should the final line retain?

The final line of `top_items` is critical. It must retain only the labels whose entries survived the threshold. Were entries equal to zero retained, every phenotype would contain every diagnosis, and all subsequent analysis would fail.

Verified properties. The threshold is relative, so that scaling a vector leaves the pattern of zeros unchanged; the input array is not modified; and only retained labels are returned, in descending order of weight.

TODO 3 — `avg_overlap`

Mean pairwise cosine similarity between components.

```
R = factor.shape[FILL_IN]
N = factor / (np.linalg.norm(factor, axis=FILL_IN, keepdims=True) + 1e-12)
S = FILL_IN.T @ FILL_IN
iu = np.triu_indices(FILL_IN, k=FILL_IN)
return float(S[iu].mean())
```

Questions. Which element of `shape` gives the number of components? Along which axis must the columns be normalised? Should the similarity matrix be computed from the raw or the normalised columns? Should the diagonal, on which each column is compared with itself, be included?

Verified properties. Identical columns yield 1.0; columns without shared entries yield 0.0; the result is independent of magnitude; and the mean is taken over all pairs.

TODO 4 — `overlap_penalty`

The penalty term favouring dissimilar components.

```
R = A.shape[1]
G = FILL_IN.T @ FILL_IN
G = G * (1 - np.eye(FILL_IN))
return FILL_IN * (A @ G)
```

Questions. Which matrix's columns are compared? What size must the identity matrix be? Which parameter determines the strength of the penalty?

Verified properties. Columns without shared entries incur no penalty; overlapping columns do; a weight of zero removes the penalty; and the penalty scales linearly with its weight.

TODO 5 — `fit_bias_and_interaction`

Estimation of the bias and interaction tensors by alternating optimisation.

```
bias = ncp(X, FILL_IN, n_iter=n_inner, seed=seed)
for _ in range(n_outer):
```

```

left_over = np.maximum(X - reconstruct(FILL_IN), 0)
inter = ncp(left_over, FILL_IN, ...)
left_over = np.maximum(X - reconstruct(FILL_IN), 0)
bias = ncp(left_over, FILL_IN, ...)
return bias, inter

```

Questions. How many components does the bias tensor comprise, and how many the interaction tensor? Which term is subtracted before the interaction tensor is estimated, and which before the bias tensor is re-estimated? Section 2.6 describes the procedure.

Verified properties. The bias tensor is rank-one; the interaction tensor has R components; the bias tensor contains the baseline conditions; and at least three of the four planted phenotypes are recovered.

5. Verification

A verification cell follows each function and should be executed before proceeding.

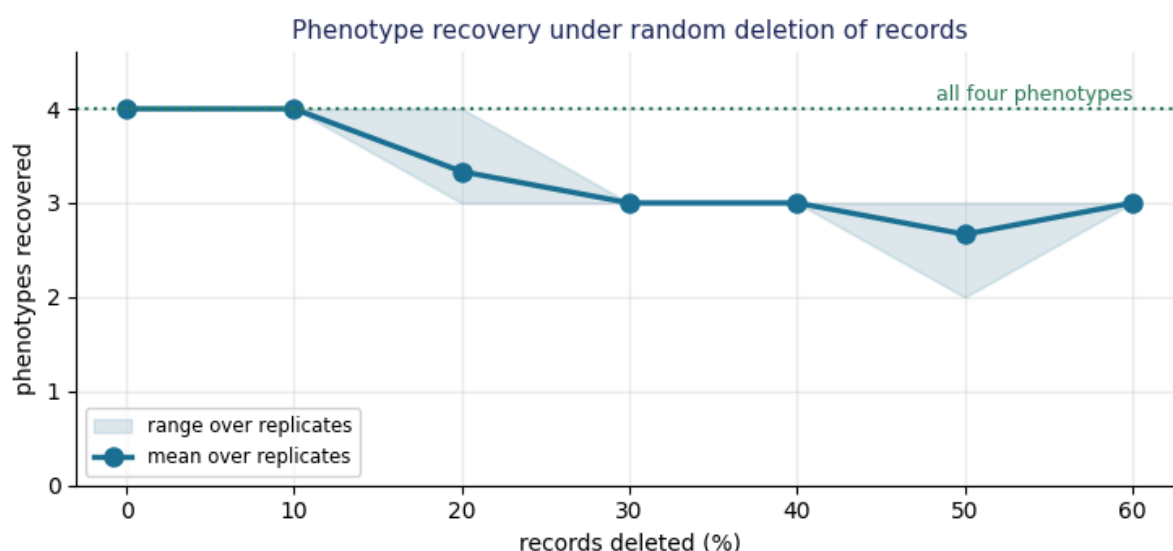
Following	Execute
loading of the tests	<code>check_all()</code> ; all tests are expected to fail at this stage
TODO 1	<code>test_reconstruct()</code>
TODO 2 and 3	<code>test_sparsify()</code> and <code>test_avg_overlap()</code>
TODO 4	<code>test_overlap_penalty()</code>
TODO 5	<code>test_fit()</code> , followed by <code>check_all()</code>

A test executed while a blank remains unfilled reports that a `FILL_IN` remains to be completed. The tests compare outputs against values derived analytically and are independent of any reference implementation; a function that passes its tests is therefore correct.

The verification cells should not be omitted. The notebook executes without error even when a function is incorrect, and produces results that appear plausible. An error detected by the tests would otherwise become apparent only much later, in a form suggesting an unrelated problem.

6. Robustness to missing records

Clinical records are incomplete. Robustness is assessed by deleting a random proportion of the recorded entries and refitting the model, with three independent replicates at each level. The line shows the mean number of phenotypes recovered; the shaded band spans the range over replicates.



Replication is necessary because the replicates do not always agree; a single replicate would not permit a genuine decline in performance to be distinguished from random variation.

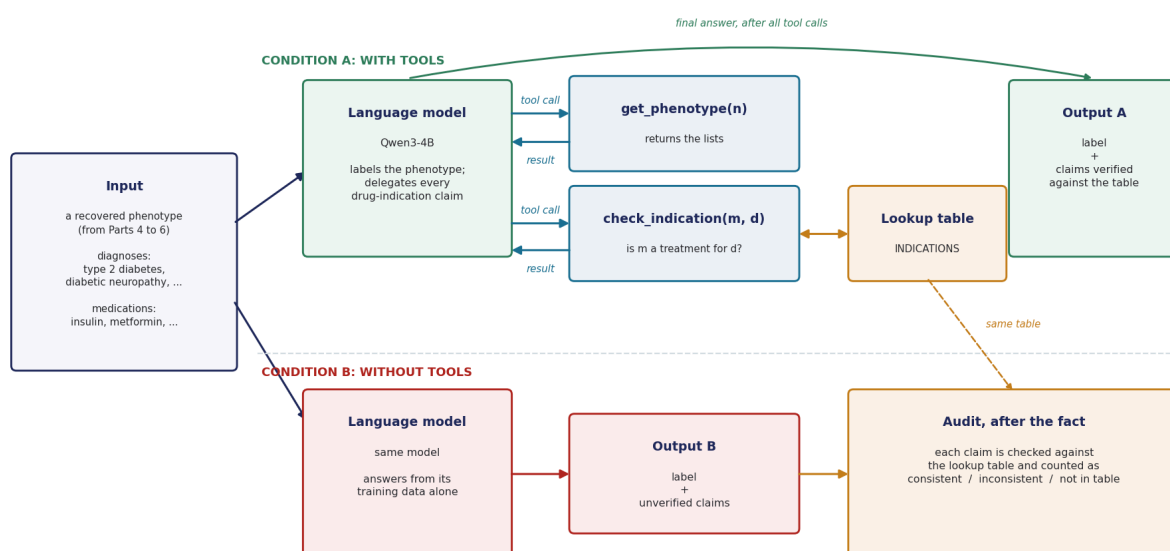
7. Annotation of phenotypes by a language model

The recovered phenotypes are lists of diagnoses and medications, which still require interpretation. In the original study this was performed by three physicians. Expert review of this kind is rigorous but does not scale, which motivates the question of whether a language model could perform part of the task.

The final part of the notebook examines this question under two conditions, summarised in the figure below. The same model and the same phenotypes are used in both; only access to external knowledge differs.

Part 8: annotation of recovered phenotypes by a language model

The same model and the same phenotypes are used in both conditions. Only access to external knowledge differs.



Compared in written question 5: the accuracy of the labels, and the number of claims that contradict the lookup table.

Not attempted: judging whether a phenotype is clinically meaningful. This would first require validation against physician raters (written question 6).

In **Condition A** the model has access to two tools: one that returns the phenotype, and one that checks a medication–diagnosis pair against a lookup table. Each claim about a drug indication is therefore

verified at the moment it is made. In **Condition B** the model answers from its training data alone, and its claims are audited against the same table afterwards.

Three uses of a language model are distinguished. Assigning a descriptive label to a phenotype is appropriate, as its output can be verified against the planted phenotypes. Asserting that a medication treats a diagnosis is inappropriate without external grounding, since the model answers confidently and occasionally incorrectly. Judging whether a phenotype is clinically meaningful is not attempted, since no ground truth exists against which such judgements could be checked.

8. Concept questions

Seven short multiple-choice questions are distributed through the notebook. Each is answered by editing a statement of the form `concept("q3", "?")` and executing it, which returns immediate feedback.

1. The total number of entries in the tensor.
2. The reason a component is allocated to hypertension and statins.
3. The representation of a condition shared by every patient.
4. The effect of setting `gamma = 1.0`.
5. The overlap between two identical components.
6. The significance of a very low overlap score.
7. The differential effect of missing records on small and large phenotypes.

9. Written questions

Six questions at the end of the notebook require extended answers of three to four sentences each.

Answers must report the results actually obtained, not the results expected. Answers describing expected rather than observed behaviour receive at most half credit.

1. In Part 3 the standard factorization allocated a component to the population baseline. Explain why the baseline constituted the dominant pattern in the data despite being the least informative.
2. Select a value of `gamma` suitable for presenting the phenotypes to a clinician. Report the resulting lists, justify the choice, and discuss what is lost and gained at a higher value.
3. In Part 6 overlap decreased as λ increased. Two of the planted phenotypes genuinely share chronic kidney disease and ACE inhibitors. Explain why a low overlap score is therefore not equivalent to a correct result.
4. In Part 7 phenotypes were lost as the proportion of deleted records increased. Identify the phenotype that was lost first and propose an explanation.
5. Compare the language model's responses with and without access to tools. Assess whether the phenotypes were labelled correctly in each condition, and whether the tool-assisted condition identified errors present in the unassisted condition. Quote one example..
6. The original study relied on ratings by three physicians. Describe the study that should be conducted before one rater could be replaced by a language model, and specify the result that would justify the replacement.

10. Assessment

Component	Points
TODO 1 reconstruct: all tests pass	8
TODO 2 sparsify and top_items: all tests pass	8
TODO 3 avg_overlap: all tests pass	8
TODO 4 overlap_penalty: all tests pass	8
TODO 5 fit_bias_and_interaction: all tests pass	13
The seven concept questions	7
Parts 1 to 8 executed, with output visible	8
Question 1: dominance of the baseline	6
Question 2: choice and justification of gamma, with lists reported	8
Question 3: distinction between low overlap and correctness	8
Question 4: identification and explanation of the first phenotype lost	6
Question 5: comparison of the two model conditions, with a quoted example	8
Question 6: study design and acceptance criterion	4
Total	100

A function that fails its tests receives no credit for that item, irrespective of how closely it approaches a correct solution. The tests may be executed any number of times before submission.

11. Troubleshooting

Problem	Resolution
A test reports an outstanding <code>FILL_IN</code>	Complete the remaining blank in the indicated function.
Every phenotype is reported as no planted phenotype matches	<code>top_items</code> is retaining entries that should be discarded. Execute <code>test_sparsify()</code> .
The language model returns (no response)	The same cause as above; correct <code>top_items</code> .

Problem	Resolution
FileNotFoundError when loading data	Upload both CSV files through the Colab file panel and execute the cell again.
“No GPU” in Part 8	Select Runtime → Change runtime type → T4 GPU, then execute the notebook from the beginning.
Insufficient memory	Select Runtime → Restart session and execute from the beginning, loading the model only once.
Slow model download	Substitute Qwen/Qwen3-1.7B and state this in the submission.
Components differ between runs	The algorithm is initialised randomly; the seed may be changed.

A GPU is required only for Part 8. All other parts execute within seconds on the free Colab tier.

11. Glossary

Phenotype	A clinically coherent subgroup of patients characterised by shared diagnoses and medications
Tensor	A multidimensional array; here, of order three
Mode	One dimension of a tensor: patient, diagnosis or medication
Order	The number of indices required to identify an entry of a tensor
Rank-one tensor	The outer product of one vector per mode
CP decomposition	The approximation of a tensor as a sum of rank-one tensors
R	The number of components in the decomposition
Factor matrix	The vectors of one mode across all components, arranged as columns
Bias tensor	The rank-one term representing the population baseline
Interaction tensor	The sum of the R components representing the phenotypes
Non-negativity	The constraint that all factor entries be non-negative, so that phenotypes are defined by the presence of features
Sparsity	The setting of small entries to zero in order to improve interpretability
gamma	The sparsity threshold, expressed as a fraction of the largest entry in a vector
Cosine similarity	A measure of the similarity in direction of two vectors
AvgOverlap	The mean cosine similarity over all pairs of components
lambda	The weight of the penalty favouring mutually dissimilar components
FILL_IN	A blank in the provided code, to be replaced by the student